

Memory Management

Laboratory 4

Duration: 3 weeks

We learnt techniques in class to (i) improve performance of memory virtualization using a TLB, and (ii) reduce space requirement of memory virtualization by using multi-level page tables. In this assignment, let us evaluate the efficacy of these techniques using some real world benchmarks.

Please find the traces corresponding to some real world benchmarks [here](#). Each line in a trace file corresponds to a single memory request generated by the pipeline – it could be instruction or data. The format of each line is `<address>, <I/D>, <R/W>`. `I/D` indicates instruction or data, `R/W` indicates read or write. Tip: the files are very large – during the development phase, work with smaller files derived from these large files.

The task is to develop a memory virtualization simulator in C++.

Assume:

- The virtual address space is of size 2^{32} Bytes (use the lower 32 bits in the addresses found in the trace files). Page size = 4 KB.
- TLB: 128 entries, fully associative, LRU policy. TLB is empty at the start of simulation.
- TLB access takes 1 cycle, while memory access takes 200 cycles.
- There are no instruction and data caches.

Part I

Assume that the number of frames is greater than the number of pages (not realistic!). All frames are free at the start of simulation.

Develop a simulator of a single-level page table. Perform virtual page to physical frame mapping. Make up your own scheme for managing free frames and allotting frames to pages. Document your scheme in the report.

Report for each benchmark, and comment on your observations:

- A. Number of frames used
- B. There is a hypothesis that a small fraction of pages see a large fraction of the accesses and vice versa. Provide an interesting plot that supports or refutes this hypothesis.
- C. Number of cycles taken to execute (sum up the time taken by each instruction)
- D. Page table utilization = number of PT entries accessed / number of PT entries allocated
- E. A hash

For every access made by the benchmark, the hash H must be updated as follows: $H = f(H, \text{frame_number})$. Choose function f on your own.

Part II

Develop a simulator for a two-level page table. Repeat the studies A-E mentioned in Part I and comment on your observations.

Does the hash computed here match the hash computed in Part I?

Part III

Add a TLB to the simulated system. Report for each benchmark, and for both the single-level and the two-level cases:

- A. Hit rate of the TLB
- B. Number of cycles taken to execute

Part IV

We made an assumption that the number of frames is greater than the number of pages. Let us remove this. Experiment with different sizes of physical memory (say 0.5 times the virtual memory size, 0.25 times the virtual memory size, and so on). Assume that a disk access costs 20 million cycles. Count disk access costs only when pages are replaced – let us assume the first set of pages come for free. Assume LRU replacement policy.

Report for each benchmark:

- A. Page fault rate
- B. Number of cycles taken to execute

Part V

Let us see who comes up with the fastest simulator. Record the time taken by your simulator for each benchmark.

Submission:

- Single zip file containing
 - Source code with suitable makefiles
 - Report containing observations in the form of graphs and their analyses.